

Helping Students with Motor Impairments Program via Voice-Enabled Block-Based Programming

Obianuju Okafor(✉) and Stephanie Ludi

University of North Texas, Denton, TX 76205, USA
{obianujuokafor,stephanie.ludi}@unt.edu

Abstract. Existing programming environments pose a challenge for students with upper-body motor impairments. This is because these environments require a level of dexterity that these students do not possess. For example, text-based programming environments require a lot of typing using a keyboard, while block-based programming environments require the use of a pointing device to drag and drop blocks of code. In our research, we aim to make the block-based programming environment Blockly, accessible to students with upper-body motor impairment, by adding speech as an alternative form of input. This voice-enabled version of Blockly will reduce the need for the use of a mouse or keyboard, hence making it more accessible. Our system consists of the original Blockly application, a speech recognition API, predefined voice commands, and a custom function. A preliminary study has been conducted. The results are encouraging, but they also revealed the need to broaden the target population, which was originally people with cerebral palsy, to people with any type of upper-body motor disability. Additionally, the study showed the need to redesign some voice commands. A prototype of the system has been implemented. As a next step, two additional studies will be conducted using this prototype, a usability study, and an A/B test.

Keywords: Accessibility · Block-based programming · CS education · Speech recognition

1 Introduction

Students with upper body motor impairments (UBMI) such as cerebral palsy, muscular dystrophy, multiple sclerosis, etc., often suffer from paralysis, muscle weakness, and poor coordination [19]. This makes performing actions like typing on a keyboard, or dragging and dropping objects using a mouse, quite hard [29]. These challenges are more apparent in environments such as schools, particularly in hands-on classes, where such actions are frequently performed. An example of such classes is the introductory programming classes, which are

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2022
M. Antona and C. Stephanidis (Eds.): HCII 2022, LNCS 13309, pp. 62–77, 2022.
https://doi.org/10.1007/978-3-031-05039-8_5

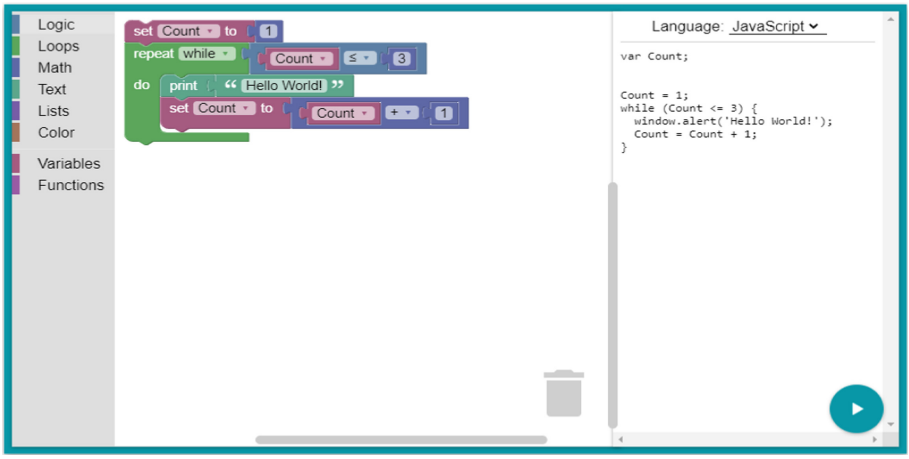


Fig. 1. User interface of the BBPE blockly

being incorporated into the curriculum across the United States as part of the **“Computer Science for All”** initiative [4,10,26]. In these classes, students are taught programming concepts and computational thinking using different programming tools [2,3,5]. Sadly, a lot of these programming tools are inaccessible to students having UBMI as they rely heavily on the use of a mouse or keyboard.

A particular set of programming tools that pose a challenge are block-based programming environments (BBPE) such as MIT's Scratch[1] or Google's Blockly[2]. In BBPEs, programs are created by dragging and dropping code blocks using a mouse or keyboard (see Fig. 1). Due to their highly engaging nature and simplicity, BBPEs are often used in curricula to teach children how to program [2,13,24,25]. However, their reliance on the physical controlling of a pointing device poses a barrier for students with UBMI. This limitation excludes a significant portion of students from participating in activities in classrooms that may allow them to explore career paths in computing.

This research addresses the obstacles that students with UBMI face in the BBPE Blockly, and seeks to come up with an effective solution. In our approach, we explore speech as an alternative input modality to increase Blockly's accessibility to students with UBMI. With the rise in the use of conversational agents, like Amazon's Alexa[3] and Apple's Siri[4], speech is rapidly becoming a popular way of interacting with computers or digital devices, even for people with motor disabilities [6,11,12,16]. This could be attributed to its convenience and speed in sending and receiving information. For these reasons,

[1] https://scratch.mit.edu/.
[2] https://developers.google.com/blockly/.
[3] https://developer.amazon.com/en-US/alexa.
[4] https://www.apple.com/siri/.

we see speech as a possible method to alleviate the challenges that students with UBMI face in BBPEs. The goal here is not to replace the mouse or keyboard as a form of input in Blockly, but to provide an alternative for those who cannot use the mouse or keyboard.

In our system, we incorporate a speech recognition API into the Blockly application. To confirm the feasibility of the voice-based system, we conducted a preliminary study. The study yielded promising results, it also exposed areas for growth. We go into details about this preliminary study in Sect. 4. Based on the feedback from the preliminary study, we have created a prototype of the proposed system, which we present in this paper. We aim to evaluate the prototype through 2 additional studies.

The main contributions of this paper are:

1. The system design of the novel voice-enabled Blockly system
2. A guide on how to incorporate speech recognition into an existing application
3. Design considerations for future voice-based systems based on analyses of user feedback.

The rest of the document is structured as follows. In Sect. 2, we talk about the works that relate to our research. In the 3rd section, we give an overview of our system. In Sect. 4, we present the experimental setup and findings of the preliminary study we conducted. In the last section, we conclude and discuss the next steps.

2 Related Work

We found several research studies relating to how speech can be used to assist people with motor challenges when interacting with computer applications and devices, and when programming.

A popular use of speech for people with motor impairments is for mouse cursor control. A lot of people with UBMI face difficulty when moving a mouse cursor or performing a variety of other continuous control tasks. Speech was considered as a possible workaround to circumvent these challenges. Dai et al. [6,18] present a speech-controlled grid-based cursor control mechanism. This mechanism was created to address the inadequacy of existing cursor control solutions, as they were found to be both slow and error prone. They evaluated two 3×3 grid-based variations of their system. One provided a single cursor in the middle of the grid, the other allows users to select a target using any of the nine cursors. They found out that the nine-cursor solution outperformed the one-cursor solution in terms of speed. Overall, both grid-based solutions performed better than other speech-based cursor control solutions.

Similarly in [11], to combat the challenges people with motor impairment face when moving a mouse cursor or performing other continuous control tasks, the authors present a system called the Vocal Joystick. This system allows users to continuously control the mouse cursor using verbal and nonverbal vocalizations,

along with varying vocal characteristics such as vowel quality, loudness, and pitch. The results of the two studies they conducted showed that Vocal Joystick performed as optimally as conventional hand-operated joysticks. The findings also showed that it can be used by people without extensive training, and that it could be an alternative to existing speech-based cursor control methods.

Speech is not only useful for cursor control, but also for text entry. In [20] Vertanen et Mckay presented a speech-based text entry system called Speech Dasher. In this system, first, users speak what they want to write and then through zooming, they navigate through the space of recognition hypotheses to correct any errors. Speech Dasher's model uses a combination of information from a speech recognizer, the user, and a letter-based language model. The formative user study that was conducted with Speech Dasher revealed that expert users were able to write 40 words per minute (wpm), and they were able to do this in spite of a recognition word error rate of 22%. This confirmed that Speech Dasher successfully utilizes information from speech recognition to greatly improve users' writing efficiency.

The use of speech for cursor control and text entry makes it particularly useful in programming environments. The authors in [17] synthesized the results of a Wizard of Oz based design process to develop VocalIDE. VocalIDE is a voice-based programming environment prototype that allows people with limited dexterity to write and edit code using a set of vocal commands. They evaluated the usefulness of VocalIDE with 8 participants who have upper limb motor impairments. Their results showed that VocalIDE improved the participants' ability to make navigational edits and select text while programming.

Desilets et al. [8], created several voice-based programming tools, one of which is VoiceCode. VoiceCode was proposed as a solution for people suffering from Repetitive Strain Injury (RSI). This tool allows developers to use naturally spoken syntax to write, navigate, and modify code. The naturally spoken commands get converted to actual code in real-time. The system was found to be useful, but it could not be effectively used to teach programming to novices or students with visual impairment. VoiceGrip is the other tool created by Désilets et al. [7]. Akin to VoiceCode, VoiceGrip enables programmers to dictate code using an easy-to-utter pseudocode, which is automatically translated to native code in the appropriate programming language. Voicegrip was created to address some of the usability problems associated with programming by voice.

The research most similar to ours is the one by Wagner et al. [21–23]. In their work, they address the challenges people with UBMI face in the BBPE Scratch. They created a voice-driven Java tool called Myna, that enables programming by voice within Scratch. This tool processes voice commands from users, interprets those commands according to a preset grammar, and simulates the synonymous actions of a mouse and keyboard within Scratch. In contrast, in our project, the block-based platform we added speech input to is Blockly, not Scratch. Also, we added speech as one of Blockly's input modalities and not as a stand-alone tool as in the case of Myna. The rationale behind this is that this will make the user experience more seamless, and will leave room for collaboration among

children with and without dexterity skills. Additionally, Myna was not evaluated by enough users, we plan to conduct 2 user studies on the voice-enabled Blockly prototype using a larger number of participants.

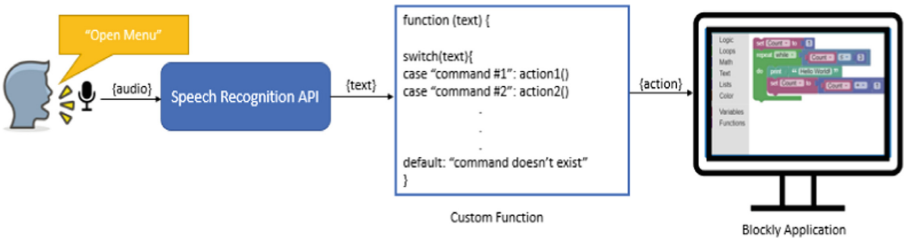


Fig. 2. System overview

3 System Overview

Our system is a voice-enabled version of the Blockly application. Originally, in Blockly, the primary form of input is a mouse or keyboard. We added speech as an additional input modality for people with UBMI. An overview of the system can be seen in Fig. 2.

The system comprises four major components; the Blockly application, speech recognition API (SRAPI), voice commands, and a custom function. These components will interact in the following way: The speech recognition API receives speech input through the device's microphone. The audio input is processed and converted to text. The generated text is sent to the custom function. The custom function checks through all switch statement cases to see if there is a voice command that matches the text. If there is a match, the action associated with that voice command is executed, which triggers a change to Blockly's user interface. We go into more details about each component in the subsections below.

3.1 Blockly Application

Blockly is a browser-based block-based programming environment (see Fig. 1). In Blockly, programs are created by dragging and dropping blocks of code using a mouse or keyboard, onto a workspace. These code blocks represent programming constructs, e.g., loops, conditional statements, variables, etc. The constructed program can be translated into code text by the Blockly application. The programming languages supported include Javascript, Python, PHP, Dart, Lua.

Blockly's source code was made available online via GitHub[5], hence the code used in this research project was downloaded from Blockly's GitHub repository.

[5] https://github.com/google/blockly.

Most of the functions in the original code were executed based on mouse clicks or keyboard presses. The code was modified so that the same set of functions can be executed using speech.

```
1  window.SpeechRecognition = window.webkitSpeechRecognition
2                           || window.SpeechRecognition;
3  let recognition = new window.SpeechRecognition();
4  recognition.lang = "en-US";
5
6
7  recognition.onresult = function (event) {
8  const transcript = event.results[event.results.length - 1][0]
9                          .transcript
10                         .toLowerCase()
11                         .trim();
12
13     custom_function(transcript);
14 };
15
16 recognition.start();
```

Fig. 3. Code showing how the speech recognition API was integrated with Blockly

3.2 Speech Recognition API

Speech recognition API (SRAPI) is a robust prebuilt library that records speech in real-time, converts it to text, and returns the text. There are several state-of-the-art SRAPIs that exist, such as CMU Sphinx[6], Google Cloud Speech API[7], and Dragon Speech[8]. The SRAPI we chose was Web speech API[9], reasons being that it is JavaScript-based like Blockly, open-sourced, and compatible with most web browsers. This API was also used in several researches [14,17].

Web Speech API was integrated with the Blockly application by adding a few lines of code to Blockly's source code. Figure 3 is a code excerpt showing how Web speech API is integrated into the Blockly application. Lines 1–4 show how it is initialized. Lines 7–14 show the function that processes speech input, converts it to text, and sends the text to the custom function. Line 15 shows how speech recognition is started.

[6] https://cmusphinx.github.io/.
[7] https://cloud.google.com/speech-to-text.
[8] https://www.nuance.com/dragon/for-developers/dragon-software-developer-kit.html.
[9] https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API.

3.3 Voice Commands

The voice commands in our system are limited and predefined. This helps prevent any ambiguity associated with more verbose speech recognition systems. Each voice command has an action that they trigger. These actions, when triggered, simulate different keyboard or mouse actions, e.g., selecting, dropping, etc. Table 1 shows all the commands in this system and their function in different modes.

These voice commands can be broken down into 5 categories:

1. Navigation Commands: These are the commands used to move through menus, drop-down menus, and in between a stack of blocks on the workspace. Using these commands, a user is able to control and move the cursor from one point to another. This set of commands are similar to the WASD keyboard shortcuts. Some examples include: *"up"*, *"down"*, *"in"*, *"out"*.
2. Placement Commands: These commands are used to select blocks on the menu and place them on the workspace. They can also be used to remove blocks from the workspace. These commands are synonymous with dragging and dropping blocks using a mouse or keyboard. Some examples include: *"select"*, *"delete"*.
3. Control Commands: These set of commands are responsible for controlling elements in the interface, such as opening and closing menus. Some examples include: *"menu"*, *"close"*.
4. Edit Commands: These commands are used to edit a block's comment or their text value. It can also be used to change the option selected from the dropdown menu. Some examples include: *"edit field"*, *"save"*.
5. Mode Commands: These commands are used to switch between the 3 modes in our system. The modes are navigation mode, connect mode, and edit mode. The default mode is **navigation mode**. In this mode, the user can move the cursor around using the navigation commands. In the **connection mode**, a user can connect blocks together on the workspace. To make edits using the edit commands, the user will enter **edit mode**. They include: *"connect"*, *"edit"*, *"escape"*.

3.4 Custom Function

Originally in Blockly, actions are performed using a mouse or keyboard. To perform actions in Blockly using speech, we created a function that maps voice commands to actions. This function entails a switch statement [28]. In the switch statement, each case is a voice command. Each voice command is paired with a corresponding action function to be executed. For instance, the voice command *'delete'* is paired with the block deletion function.

This custom function takes as input the text generated by the speech recognition API (see Fig 3). This text is compared to all cases in the switch statement to determine what action to perform. If there is a match, the action associated with that command is executed. If there is no match, no action is performed, also, the message "command doesn't exist" is displayed to the user in the console.

Table 1. Voice commands and their function in the different modes

Command	Navigation mode	Connection mode	Edit mode
Menu	Go to first item on menu	–	–
Up	Move up to the previous item on the menu or to the previous block on the workspace	Move to the previous connection point	–
Down	Move down to the next item on the menu or to the next block on the workspace	Move to the next connection point	–
In	Move into the inner menu or move from an outer block (parent) to an inner block (child)	Move to the next connection point	–
Out	Come out of the inner menu or move from an inner block (child) to an outter block (parent)	Move to the previous connection point	–
Inline	Move to a block on the same line as the block you are currently on	-	–
Select	Select a block on the menu	Select a connection point	Select an option in a drop down menu
Connect	Switch to connection mode where you can connect blocks together	–	–
Edit	Switch to edit mode where you can make changes to text fields or dropdown menus	–	–
Edit field	–	–	Edit selected text field or dropdown menu
Escape	–	Switch back to navigation mode	Switch back to navigation mode
Add comment	Create a comment for selected block	–	–
Save	–	–	Save new text field input or new comment text
Delete	Delete selected block from the workspace	–	–
Undo	Undo last action	–	–
1–9	Quick jump to menu category according to their position	–	–
First	Jump to first block in the workspace	–	–
Last	Jump to last block in the workspace	–	–

Table 2. Participant information

Participant ID	Age	Gender	Assistive technology used
P1	40	Male	ECO2 augmentative communication device
P2	68	Female	None
P3	68	Female	None
P4	34	Male	None
P5	66	Female	None

4 Preliminary Study

Before implementing the system, we conducted a preliminary study. The study aimed to confirm the feasibility of the use of voice as an input modality, particularly for people with cerebral palsy who were our target audience at the time, as they are known to sometimes have speech impediments [9,15]. The study also helped us to test the appropriateness of our voice commands. To achieve our research goals, we recruited participants with cerebral palsy to read out commands to the selected speech recognition API, Web Speech API.

4.1 Participants

6 people with cerebral palsy agreed to participate in the study, however, on the day of the study, only 5 of them participated. The participants comprised of 3 females and 2 males. The average age was 55.2 years (SD = 14.99). We got in contact with them through a nonprofit organization called Ability Connection [1], which they were members of. This organization caters to people with intellectual or developmental disabilities. Table 2 shows information about the participants.

4.2 Experimental Procedure

The study was conducted in person in a vacant room at the Ability Connection facility. In the experiment, the participants voiced 10 commands to the demo interface of the speech recognition API, Web Speech API. They interacted with this system via a demo interface[10] which was made available online by Google. Figure 4 shows a screenshot of this interface. Each participant was met individually. Each session lasted about 30–45 min. At the end of the study session, each participant was given a $20 Amazon gift card.

At the start of each session, introductions were made and the participant was asked to choose an alias they would like to be addressed as. The participants were all excited about this and often chose the name of a character from their favorite movie or show. Next, the task to be performed during the study was

[10] https://www.google.com/intl/en/chrome/demos/speech.html.

described. It was also made known to each participant that they can pause or stop at any time if the task becomes too cumbersome for them. Most of the participants did not own a laptop, so we used one of our laptops for the study.

To start the experiment, the participant was handed a list of 10 commands (see the first column of Table 4) and then asked to read each one out to the speech recognition interface which was opened up in a browser on the laptop. For each command, they had 3 attempts. The participants were observed as they read each command. For each command, the participants were given a score according to how well the system is able to recognize the command.

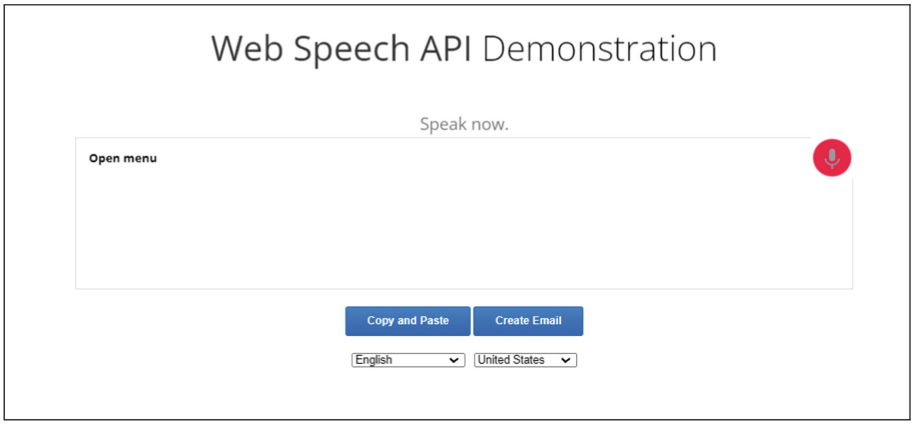


Fig. 4. Demo interface of web speech API

At the end of the study, the participant was given a survey. The survey had a total of 10 questions. Some of the questions include; age, assistive technology (if any), their overall experience while using this speech recognition system, the difficulty of pronouncing each command, what commands they liked, what commands they disliked, what they would like to change, etc. As most of them could not type or write, they were asked each question and their responses were written down. Table 4 shows a summary of some of the survey responses.

4.3 Result

In this section, we present the results of the study. We break the results down into two sections, the performance of the speech recognition system and the responses from the survey.

Speech Recognition System Performance. Word error rate (WER) and word accuracy (WAcc) are common metrics used to rate the performance of a speech recognition system [27]. In this study, we would be using WAcc. The

WAcc score represents how well the system is able to recognize each command the participant vocalized. Table 3 shows the WAcc per command for each participant. The WAcc for each command is added to form a total WAcc. There were 10 commands in the task, each command weighed 1 point, therefore, the maximum total WAcc a participant could have was 10. The higher the WAcc score, the better the performance.

We calculated the WAcc as follows: for each command, the system was able to recognize completely, 1 point was awarded, else points were deducted accordingly. For each attempt failed, extra points were deducted. You can see the formula for WAcc below:

$$WAcc = 1 - WER$$
$$WER = \frac{U + I}{N} + F(0.1)$$

where:

N: number of words in a command
U: number of undetected words in a command
I: number of incorrect words in the command
F: number of failed attempts

Table 3. Participant WAcc score per command

PID	Command #										
	1	2	3	4	5	6	7	8	9	10	Total WAcc
P1	1	1	0.8	0.9	1	1	0.9	0.9	1	1	9.5
P2	1	1	1	1	1	1	1	0.9	1	1	9.9
P3	1	0.6	1	1	0.9	0.6	0.8	0.7	0.6	0.3	7.5
P4	0	0	0.7	0	0	0	0.6	0	0	0	1.3
P5	0.9	0.8	0.3	0.7	0.9	0.9	0.6	0	0.3	0	5.4

The mean score for WAcc was 6.66 (SD = 3.5). The maximum total WAcc score of 9.9 was by P2, who had no speech impediment at all, so the speech recognition system could understand her perfectly. She also took the least time to complete tasks. The second highest total WAcc score was by P1. He scored a 9.5. He could not speak, but he used an ECO2 Augmentative Communication device to vocalize the 10 commands. His session took the longest as he had to type each command. Due to his limited dexterity, he sometimes hit the wrong key and had to erase the entire text and type it again. However, out of all participants, he was the only one who had some programming experience.

Occasionally, the system failed to detect speech and this affected the WAcc of P3 and P5 who had a total WAcc of 7.7 and 5.4, respectively. They both

spoke coherently, however, they had a peculiar accent which sometimes made it hard for the speech recognition system to detect their speech. A lot of times they had to repeat a command multiple times. Additionally, P5 spoke in a low voice. For the system to detect her voice, she was told to speak more loudly, however, there was a limit to how loud she could speak. This further exacerbated her score, giving her the second to the lowest total WAcc score.

Additionally, the system performed poorly when the user's speech was slurred, due to this, P4 had the lowest cumulative WAcc of 1.3. The system was only able to partially recognize 2 out of the 10 commands he pronounced. Although the system was not able to detect his speech, he was still comprehensible. He gave a great suggestion which we will discuss in the next section.

Table 4. Total number of likes, dislikes, and average difficulty rating of each command

Command	No. of likes	No. of dislikes	Average difficulty rating (1–5)
1. Move Up	3	1	1.5
2. Move Down	2	0	2
3. Move In	1	1	1.75
4. Move Out	0	0	1.25
5. Open Menu	2	0	2
6. Close Menu	0	2	2.25
7. Select Block	1	1	2
8. Select First Block	1	3	3
9. Delete Block	0	1	2.75
10. Edit Mode	2	2	2

Survey Responses. We present a summary of some of the survey responses. Table 4 shows the total number of likes and dislikes each command received, as well as its average difficulty rating.

According to the survey responses, 60% rated their experience using the speech recognition system as positive. The rest of the participants rated their experience as average. Some reported that they enjoyed interacting with the speech recognition system and even stated they would like to do it again.

When asked what command they liked, over half of the participants stated that they liked the first command *"move up"*. This could be because it was the first command on the list. The second most liked command were these three commands; *"move down"*, *"open menu"* and *"edit mode"*. In particular, P3 said she liked the sound of *"edit mode"*, and she giggled whenever she said it. In contrast, the most disliked command was the command *"select first block"*. It was also rated as the most difficult to pronounce. Based on observation, this

was also the command that most participants struggled to pronounce. Perhaps this could be attributed to the fact that it was the lengthiest. The second most difficult command to pronounce according to the ratings was *"delete block"*.

When asked to give comments or suggestions on how to improve, P4 gave a great suggestion. He suggested that some of the commands should be shortened to enable people with speech impediments to pronounce them more easily. For instance, *"move up"* could be changed to *"up"*, *"select block"* to *"select"*, and *"select first block"* to *"select first"* or *"first"*. This suggestion was insightful, as based on observation, the longer the command, the harder it was for the participants to pronounce, and the harder it was for the system to recognize what the participant was saying. P2 also gave the same suggestion as P4.

4.4 Discussion

The results are promising, 80% of participants had an above-average performance (mean WAcc = 6.66), of which one of the participants used an assistive device to vocalize code. Some of the participants would have had a higher score, if not for glitches on the part of the speech recognition system. The participants who performed well above average without any assistance had little to no speech impairment. Their ability to voice commands implies that potentially they could use voice to write programs in the block-based programming environment Blockly. The participants' positive reaction to using the speech recognition system also reinforces this notion.

Although the results are encouraging, it revealed that not all people with cerebral palsy can use a speech recognition system. In particular, people with speech impediments struggled when enunciating commands to the system. They struggled even more when the command was lengthy or the words in the command were polysyllabic. For instance, the command *"select first block"* which was the longest command in the experiment, proved difficult for all participants except one. When a command is difficult to pronounce, it ends up being mispronounced or slurred, which makes it harder for the system to decipher what is being said. Additionally, the system also had problems detecting speech in low voices and atypical accents.

Some workarounds have to be implemented to overcome some of these barriers to adoption posed by the speech recognition system. An adjustment could be the use of a noise-canceling headset with a microphone when interacting with the system. This will help to reduce noise and increase the volume of the user's voice, which could improve the system's accuracy when capturing speech. Another hack would be having each user train the speech recognition system with their voice before they start using it. That way, the system is accustomed to the way the user will pronounce different words. Windows and IOS devices do something similar, where they have a user repeat several phrases when setting up speech recognition on their device. A more general solution would be training the system on different types of accents; that way it is robust enough to handle the different ways people pronounce different words.

Based on the findings, we decided to make some changes to our research project. The biggest modification to be made lies in the expansion of the target population of this tool. The new population is no longer limited to students with cerebral palsy, but it is open to any student with a UBMI. This is because there is a significant portion of people with cerebral palsy who cannot speak and cannot benefit from this application. Therefore, there is a need to broaden the target population to enable more people to benefit from this application. The qualifying condition for any user of the system will be that they have a UBMI and they do not have a heavy speech impediment.

Another change we made based on the feedback received, was to reform some of the voice commands. We reduced the number of words in the commands to one or at most two. This will make it easier for people with speech impairments to pronounce, it will also reduce the time spent when performing tasks. We also made sure that each command was distinct enough to enable the speech recognition system to recognize it easily. Some examples of commands that were changed include:

- Move Up → Up
- Move Out → Out
- Select Block → Select
- Delete Block → Delete

5 Conclusion

In this paper, we present our research problem which entails the challenges that students with UBMI face in block-based programming environments such as Blocky. As a solution, we propose the use of speech as an alternative form of input in the Blockly platform. In our approach, we integrated a speech recognition API into the Blockly environment.

To confirm the feasibility of a voice-based system, we have conducted a preliminary study. Although the findings from the study are auspicious, they revealed areas of improvement such as the expansion of our intended users and the modification of some voice commands. These issues when addressed, could significantly enhance the overall outcome of this research project. We took the findings from the preliminary study into account as we implemented the prototype of the system.

As a future work, this prototype will be tested on our target population through a usability study. Based on the findings from the usability study, we will make the necessary updates and finalize the implementation of the end system. Finally, this system will be evaluated alongside the original Blockly system in an A/B test.

References

1. Ability connection. https://abilityconnection.org/. Accessed 13 Jan 2022
2. Hour of Code: Hour of code activities. https://hourofcode.com/us/learn
3. Code.org: Projects. https://studio.code.org/projects/public
4. CSforALL: About csforall—csforall. https://www.csforall.org/about/. Accessed 13 Jan 2022
5. CSforALL: Curriculum directory. https://www.csforall.org/projects_and_programs/curriculum_directory/. Accessed 13 Jan 2022
6. Dai, L., Goldman, R., Sears, A., Lozier, J.: Speech-based cursor control: a study of grid-based solutions, pp. 94–101, January 2004. https://doi.org/10.1145/1028630.1028648
7. Desilets, A.: Voicegrip: a tool for programming-by-voice. Int. J. Speech Technol. **4**, June 2001. https://doi.org/10.1023/A:1011323308477
8. Desilets, A., Fox, D., Norton, S.: Voicecode: an innovative speech interface for programming-by-voice, pp. 239–242, April 2006. https://doi.org/10.1145/1125451.1125502
9. Centers for Disease Control, Prevention: What is cerebral palsy? https://www.cdc.gov/ncbddd/cp/facts.html. Accessed 13 Jan 2022
10. The New York City Department of Education: Computer science for all. https://sites.google.com/strongschools.nyc/cs4all/
11. Harada, S., Landay, J.A., Malkin, J., Li, X., Bilmes, J.A.: The vocal joystick: evaluation of voice-based cursor control techniques. In: Proceedings of the 8th International ACM SIGACCESS Conference on Computers and Accessibility, Assets 2006, pp. 197–204. Association for Computing Machinery, New York (2006). https://doi.org/10.1145/1168987.1169021
12. Harada, S., Wobbrock, J., Landay, J.: Voicedraw: a hands-free voice-driven drawing application for people with motor impairments, pp. 27–34, January 2007. https://doi.org/10.1145/1296843.1296850
13. Humble, N.: Developing computational thinking skills in k-12 education through block programming tools. In: ICERI2019 Proceedings, pp. 4865–4873. 12th annual International Conference of Education, Research and Innovation, IATED, 11–13 November, 2019. https://doi.org/10.21125/iceri.2019.1190
14. Lin, P., Van Brummelen, J., Lukin, G., Williams, R., Breazeal, C.: Zhorai: designing a conversational agent for children to explore machine learning concepts. Proceedings of the AAAI Conference on Artificial Intelligence 34, pp. 13381–13388, April 2020. https://doi.org/10.1609/aaai.v34i09.7061
15. National Institute of Neurological Disorders, Stroke: Cerebral palsy: Hope through research. https://www.ninds.nih.gov/Disorders/Patient-Caregiver-Education/Hope-Through-Research/Cerebral-Palsy-Hope-Through-Research. Accessed 13 Jan 2022
16. Oviatt, S.: Multimodal interactive maps: designing for human performance. Hum.-Comput. Interact. **12**(1), 93–129 (1997). https://doi.org/10.1207/s15327051hci1201
17. Rosenblatt, L., Carrington, P., Hara, K., Bigham, J.: Vocal programming for people with upper-body motor impairments, pp. 1–10, April 2018. https://doi.org/10.1145/3192714.3192821
18. Sears, A., Lin, M., Karimullah, A.S.: Speech-based cursor control: understanding the effects of target size, cursor speed, and command selection 2, pp. 30–43 (2002). https://doi.org/10.1007/s10209-002-0034-6

19. Society, I.N.: Motor impairment, April 2012. https://www.neuromodulation.com/motor-impairment. Accessed 12 Jan 2022
20. Vertanen, K., MacKay, D.: Speech dasher: fast writing using speech and gaze. In: CHI (2010)
21. Wagner, A., Gray, J.: An empirical evaluation of a vocal user interface for programming by voice. Int. J. Inf. Technol. Syst. Approach **8**, 47–63 (2015). https://doi.org/10.4018/IJITSA.2015070104
22. Wagner, A., Gray, J.: An Empirical Evaluation of a Vocal User Interface for Programming by Voice, January 2017. https://doi.org/10.4018/978-1-5225-1759-7.ch012
23. Wagner, A., Rudraraju, R., Datla, S., Banerjee, A., Sudame, M., Gray, J.: Programming by voice: a hands-free approach for motorically challenged children, May 2012. https://doi.org/10.1145/2212776.2223757
24. Weintrop, D.: Block-based programming in computer science education. Commun. ACM **62**(8), 22–25 (2019)
25. Weintrop, D., Wilensky, U.: To block or not to block, that is the question: Students' perceptions of blocks-based programming. In: Proceedings of the 14th International Conference on Interaction Design and Children, IDC 2015, pp. 199–208. Association for Computing Machinery, New York (2015). https://doi.org/10.1145/2771839.2771860
26. Whitehouse.gov: Computer science for all. https://obamawhitehouse.archives.gov/blog/2016/01/30/computer-science-all
27. Wikipedia Contributors: Word error rate - Wikipedia, the free encyclopedia (2020). https://en.wikipedia.org/wiki/Word_error_rate. Accessed 10 Feb 2022
28. Wikipedia contributors: Switch statement – Wikipedia, the free encyclopedia (2021). https://en.wikipedia.org/w/index.php?title=Switch_statement&oldid=1062689945. Accessed 14 Feb 2022)
29. Williamson, J.: What i've learned about motor impairment, June 2017. http://simpleprimate.com/blog/motor. Accessed 12 Jan 2022